A Computer Vision Project Report

MIDTERM PROJECT REPORT

Submitted as a requirement for the course
Computer Vision

Group Project Specialization

Object Sorting

Prepared by:

Alba, John Raymond S.
Bacsain, Jave A.
Bagaporo, John Louie B.
Nonato, Francis Gabriel F.
Palacios, Harvey Lloyd V.
Prestado, Marc Justin N.

BS Computer Science – 4A

Midterm_Object_Sorting_RealWaste.ipynb

Submitted to:

Ms. Elena Clarexe R. Gulane
Course Instructor

1. INTRODUCTION

1.1 Project Overview

This project develops the computer vision component of a vision-based autonomous 4WD robot assigned to the Object Sorting specialization. Currently, the proposed system asks one question: can the system itself see and recognize objects in front of the camera? Given an image or a live camera frame of a waste item, the system must identify which type of material it is and show that result visibly on screen. The robot is not required to move at this stage. In the Final stage, the recognized class will drive the robot's decision and motor control so that it can physically sort objects.

The Computer Vision problem addressed in this project is a multi-class image classification. We train a convolutional neural network (CNN) on the RealWaste dataset (Single et al., 2023), sourced from UC Irvine Machine Repository, a public collection of 4,752 photographs of waste items taken at the point of reception at a landfill in Wollongong, Australia. Unlike studio-quality datasets, these images show objects in cluttered, uneven, real-life conditions, which makes them a more realistic test of whether a classifier can handle the visual variation a sorting robot would face.

1.2 Problem Description

Sorting waste by material is a repetitive task that is slow, inconsistent when done by hand, and unpleasant or unsafe for the people doing it. Mistakes contaminate recyclable material and reduce its value. An automated sorter first needs a reliable way to tell what an object is from a camera image alone.

This is difficult for a computer vision system for several reasons. Items of the same class vary widely in shape, size, color and condition (a crushed bottle and an intact bottle are both plastic). Different classes can look alike (paper and cardboard share color and texture; clear glass and clear plastic reflect light similarly). Items may also be partly hidden, deformed, dirty, or photographed under changing lighting and backgrounds.

The current problem statement for this project would be: given a single RGB image of one waste item, the system must assign it to exactly one of the selected classes, report a confidence score, and generalize to images it has not seen during training.

1.3 Project Objectives

General objective:

To design, train and evaluate a computer vision system that recognizes selected waste object classes from images and live camera input, forming the vision foundation of an object-sorting robot.

Specific objectives:

- To prepare a dataset of five waste classes with at least 100 images per class, divided into separate training, validation and testing sets.
- To apply image preprocessing (resizing, color conversion, normalization) and data augmentation to prepare images for the model.
- To train a CNN classifier using transfer learning and tune it using the validation set.
- To evaluate the model on the held-out test set using accuracy, precision, recall, F1-score and a confusion matrix, with a target of at least 90% overall test accuracy.
- To demonstrate the complete pipeline (Camera/Image → Preprocessing → Model → Recognition → Visual Output) on images chosen by the instructor and on live camera input, displaying the predicted class label and confidence score.
- To check generalization on additional images captured by the group under different lighting, backgrounds and viewing angles that were not used in training.

1.4 Scope and Limitations

The current project progress covers the Computer Vision stage of Object Sorting for the Midterm. It does not cover robot motion or sorting mechanics.

- **Classes/categories.** Five material classes from the RealWaste dataset: Cardboard, Glass, Metal, Paper and Plastic. The dataset contains nine classes; the other four (Food Organics, Miscellaneous Trash, Textile Trash and Vegetation) are excluded to remain within the Midterm limit of three to five classes. The subset contains 3,092 images, well above the minimum of 100 per class.

Class	Total images	Training (70%)	Validation (15%)	Testing (15%)
Cardboard	461	323	69	69
Glass	420	294	63	63
Metal	790	553	118	119
Paper	500	350	75	75
Plastic	921	644	139	138
Total	**3,092**	**2,164**	**464**	**464**

Table 1. Class distribution and 70/15/15 stratified split of the RealWaste subset (as executed; see Section 3.5).

- **Objects/targets**. A single waste item per image, belonging to one of the five classes above, presented reasonably close to the camera.
- **Visual tasks.** Image classification: predicting the class label and a confidence score for the whole image.
- **Operating scope**. Still images from the dataset, instructor-provided images, and live camera input of items held or placed in front of the camera in an indoor setting with reasonable lighting. Processing runs on a laptop/PC.

Relevant limitations.

- The training images come from one landfill facility and one camera setup, so accuracy may drop on images with very different backgrounds, lighting or camera quality.
- The system can only output one of the five trained classes. An object outside these classes (for example food waste or fabric) will still be forced into the closest of the five.
- The dataset is imbalanced (Plastic has 921 images and Glass 420), which may bias predictions toward larger classes unless handled during training.
- Visually similar classes (Paper and Cardboard, Glass and Plastic) are expected to be the main source of errors.

1.5 Group Project Specialization

The group's assigned specialization is **Object Sorting**. The required Computer Vision task is to recognize the selected object classes so that a robot can later sort them. The system must correctly identify the selected classes from images and live camera input and show a visible result (class label and confidence score). The recognition result will be used for decision logic and motor control to move and sort objects autonomously.

2. SYSTEM DESIGN AND ARCHITECTURE

2.1 Overall System Architecture

Figure 1 shows the system architecture of our project. Part A is the pipeline run at demonstration time: an input image or camera frame is preprocessed, passed to the trained CNN, converted to a class prediction with a confidence score, and displayed on the frame. Part B is the offline pipeline that produces the trained model: the dataset subset is split, preprocessed and augmented, used to train the network, and finally evaluated on the test set.

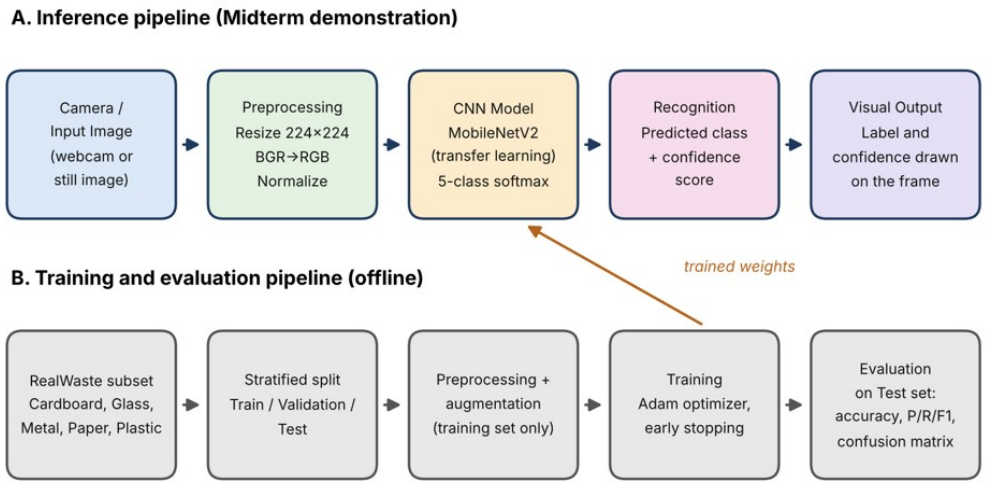


Figure 1. System Architecture for Object Sorting

2.2 Computer Vision Workflow

The Computer Vision system performs the following sequence of operations:

1. **Dataset preparation.** Match the five class names to their image folders inside the extracted RealWaste zip (524 × 524 pixel color images); images are filtered by file extension only (no explicit corruption check beyond decoding during preprocessing).
2. **Dataset splitting.** Divide each class into training, validation and testing sets using a stratified 70/15/15 split. The split is done before augmentation so that no test or validation image (or a modified copy) is seen during training.
3. **Preprocessing.** Resize every image to 224 × 224 pixels, ensure RGB channel order, and scale pixel values to the range expected by the network.
4. **Data augmentation (training set only).** Apply random flips, rotations, zoom, brightness and contrast changes so the model learns to handle different lighting and viewing angles.
5. **Model construction.** Use MobileNetV2 pre-trained on ImageNet as a feature extractor, followed by global average pooling, dropout, and a dense softmax layer with five outputs.
6. **Training.** Train the new classification layers with the Adam optimizer and sparse categorical cross-entropy loss (integer class labels), apply class weights to reduce the effect of class imbalance, then fine-tune the last 40 layers of the backbone at a lower learning rate. EarlyStopping (patience 4) and ReduceLROnPlateau watch validation loss; in the completed run neither stage stopped before its configured epoch budget (12/12, then 10/10), but EarlyStopping(restore_best_weights=True) still restores the best validation-loss checkpoint when training ends; see Section 4.4.
7. **Evaluation.** Run the final model once on the untouched test set and compute accuracy, per-class precision, recall and F1-score, and a confusion matrix. An optional cell for group-captured unseen-situation images is included in the notebook but was not executed before this report (no photo set was prepared yet); see Section 6.5 for the real-world testing that was actually performed.
8. **Recognition and visual output.** For a new image or live camera frame, apply the same preprocessing, run the model, take the class with the highest softmax probability, and draw the class label and confidence score on the image. Predictions below a confidence threshold are marked as uncertain.

2.3 Hardware Components

Listed below are the proposed hardware components relevant to the Computer Vision prototype.

Component	Specification / Model	Purpose
Camera	USB webcam, 720p (or laptop built-in camera)	Captures live frames of the object to be recognized
Controller	Laptop/PC	Runs preprocessing and model inference; later sends motor commands

Power Source	Laptop power / battery pack	Powers the processing unit
Others	Sample waste items (bottle, can, paper, cardboard, glass); plain background; adequate lighting	Live demonstration and capture of unseen test images

2.4 Software and Development Tools

The system is implemented in Python using open-source libraries. All external datasets, libraries and pre-trained models are acknowledged in the References.

Software / Tool	**Version**	**Purpose**
Python	3.11.9	Main programming language for data preparation, training, evaluation and the demonstration program
TensorFlow	2.21.0	Deep learning framework. Loads and resizes images (tf.data, tf.image), trains the CNN and runs predictions
Keras (part of TensorFlow)	3.15.1	High-level API used to build the model (augmentation layers, pooling, dropout, softmax), compile it, train it with EarlyStopping/ReduceLROnPlateau, and save it as .keras
MobileNetV2 (pre-trained on ImageNet)	Keras Applications, 224×224, include_top=False	Pre-trained CNN backbone used for transfer learning and fine-tuning
OpenCV (opencv-python)	4.14.0 (as printed by the notebook at run time)	Reads images, converts BGR→RGB, resizes for the demo, and draws the label, confidence and box on the output image. Also supports webcam frames via cv2.VideoCapture
NumPy	2.4.6	Array math: labels, probabilities, argmax, normalizing the confusion matrix
pandas	2.2.3	Tables for class counts, the train/val/test split and the classification report. Exports split.csv
Matplotlib	3.11.1	Plots the class distribution, sample images, augmentation preview, training curves, confusion matrix and prediction results
scikit-learn	1.9.1	Stratified train/val/test split (train_test_split), balanced class weights, and evaluation

		(accuracy, classification report, confusion matrix)
RealWaste dataset	UCI ML Repository, 2023 (DOI 10.24432/C5SS4G, CC BY 4.0)	Image dataset; 5 classes are used (Cardboard, Glass, Metal, Paper, Plastic), 3,092 images
Python standard library (os, pathlib, zipfile, shutil, json, random, time, glob)	Comes with Python 3.11.9	File handling, extracting the dataset zip, random seeds, timing, saving results.json and class_names.json

3. DATASET

3.1 Dataset Description

The Computer Vision system is developed and evaluated on RealWaste (Single et al., 2023), a public dataset of 4,752 photographs of waste items taken at the point of reception at a landfill in Wollongong, Australia. The dataset provides 524 × 524 pixel RGB color images organized into 9 material classes. Because the photographs were taken under real landfill conditions rather than a controlled studio setup, items appear cluttered, at varied distances, and under uneven lighting, which the group selected deliberately since a sorting robot will face the same kind of visual variation.

3.2 Target Classes / Categories

The group uses 5 of the 9 RealWaste classes, which satisfies the Midterm limit of 3–5 classes and exceeds the Specialization 8 (Object Sorting) minimum of 4 classes.

Class / Category	**Description**	**Number of Samples**
Cardboard	Corrugated/flat cardboard pieces and boxes	461
Glass	Glass bottles and jars (clear and colored)	420
Metal	Metal cans and metal containers	790
Paper	Paper sheets, cups, and paper packaging	500

Plastic	Plastic bottles, containers, and packaging	921
Total		3,092

Table 2. Target classes and sample counts for the selected RealWaste subset.

3.3 Dataset Collection and Source

RealWaste is sourced from the UC Irvine Machine Learning Repository (https://doi.org/10.24432/C5SS4G), released under a CC BY 4.0 license by Single, Iranmanesh, and Raad (2023). The notebook (src/training/Midterm_Object_Sorting_RealWaste.ipynb) downloads the dataset zip directly from the UCI static URL at run time, so no copy of the images is stored in this repository; see dataset/README.md for the exact source link and citation.

3.4 Dataset Distribution

The selected 5-class subset is imbalanced: Plastic (921 images) has about 2.2× as many images as Glass (420), the smallest class (see Table 2 in Section 3.2 for the full per-class counts). This imbalance is handled during training with class weights computed by sklearn's compute_class_weight('balanced', ...); see Section 4.4.



Figure 2. Sample images from each of the 5 RealWaste classes used in this project.

3.5 Training, Validation, and Testing Sets

The 3,092 selected images are split 70% training / 15% validation / 15% testing using a stratified split (sklearn train_test_split, seed 42) so each split preserves the same per-class proportions. The split is performed on file paths before any augmentation, and the notebook asserts that no path appears in more than one split. See Table 1 (Section 1.4) for the full per-class breakdown: 2,164 training / 464 validation / 464 testing images.

- **Training set (2,164 images).** used to update the model's weights during both training stages.
- **Validation set (464 images).** used only to monitor validation loss for EarlyStopping / ReduceLROnPlateau; never used to update weights directly.
- **Testing set (464 images).** held out completely and evaluated exactly once, after training is finished, to report the accuracy and other metrics in Section 6.

3.6 Dataset Preparation and Preprocessing

Images are decoded as RGB with tf.io.decode_image and resized to 224×224 with antialiasing (tf.image.resize). Pixel values are kept as float32 in the notebook's data pipeline; the rescaling to the [-1, 1] range MobileNetV2 expects is performed inside the model itself (a Rescaling(1/127.5, offset=-1) layer), so the same raw-pixel input works for both training and the recognize() demo function. Data augmentation is applied to the training set only: random horizontal/vertical flip, rotation (±15%), zoom (±15%), brightness (±20%), and contrast (±20%), so the model sees more lighting and viewing-angle variation without changing the validation/testing data used for evaluation.

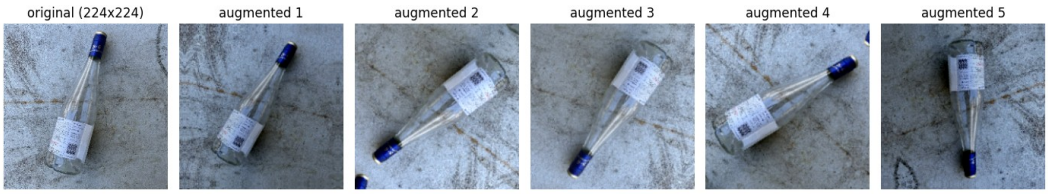


Figure 3. One training image (left) and five randomly augmented versions used only during training.

4. COMPUTER VISION METHODOLOGY

4.1 Computer Vision Approach

The group uses supervised image classification via transfer learning rather than a classical (non-learning) CV algorithm or training a CNN from scratch. With only 3,092 images across 5 classes, training a deep CNN from scratch risks overfitting; starting from MobileNetV2 weights pre-trained on ImageNet gives the model general visual features (edges, textures, shapes) that are then adapted to the waste-sorting task. MobileNetV2 is also a lightweight architecture, which keeps inference fast enough for the live recognize() demo.

4.2 Image Processing / Preprocessing

See Section 3.6 for the full preprocessing pipeline (resize to 224×224, RGB decoding, in-model rescaling to [-1, 1], training-only augmentation). Training and evaluation resize with tf.image.resize(antialias=True) through tf.data; the recognize() demo function resizes the same way conceptually but with OpenCV (cv2.cvtColor + cv2.resize, INTER_AREA) instead, since it works on one image or camera frame at a time rather than a tf.data pipeline. Both paths decode RGB, resize to 224×224, and rely on the same in-model rescaling, but the exact resize implementation differs between the two, which is a source of small numeric difference between a tf.data evaluation and a recognize() call on the same image.

4.3 Model / Algorithm

The model (build_model() in the notebook) is a MobileNetV2 backbone (keras.applications.MobileNetV2, ImageNet weights, include_top=False, 224×224×3 input) used as a feature extractor, followed by:

- **GlobalAveragePooling2D:** collapses the backbone's feature maps to one vector per image.
- **Dropout(0.3):** randomly zeroes 30% of the pooled features during training, to reduce overfitting.
- **Dense(5, activation='softmax'):** the classification head producing a probability for each of the 5 classes.

4.4 Model Training / Development

Training runs in two stages, both using the Adam optimizer and sparse categorical cross-entropy loss, with class weights applied in every epoch of both stages:

Setting	Backbone	Epoch budget	Learning rate
Stage 1 (head)	frozen	up to 12 (EarlyStopping, patience 4)	1e-3
Stage 2 (fine-tune)	last 40 layers unfrozen (BatchNorm frozen)	up to 10 (EarlyStopping, patience 4)	1e-5

Table 3. Training hyperparameters for the reported run.

EarlyStopping (patience 4, restore_best_weights=True) and ReduceLROnPlateau (factor 0.5, patience 2) watch validation loss in both stages, so the kept weights are the best-validation-loss checkpoint, not necessarily the last epoch.

4.5 Model Parameters and Configuration

Parameter	Value	Notes
Classes	5 (Cardboard, Glass, Metal, Paper, Plastic)	fixed order used for argmax → label
Input size	224 × 224 × 3	matches MobileNetV2's expected input
Batch size	32	
Random seed	42	used for the split and all shuffling
Pre-trained weights	ImageNet	keras.applications.MobileNetV2
Fine-tuned layers	last 40 layers of the backbone	BatchNorm layers kept frozen
Class weighting	sklearn compute_class_weight('balanced')	applied in both training stages
Confidence threshold	0.60	predictions below this are shown as “Uncertain”

Table 4. Model parameters and configuration used by the final implementation.

4.6 Detection / Recognition Process

Camera Frame / Image → BGR→RGB conversion → resize to 224×224 (cv2.INTER_AREA) → model.predict() → take the class with the highest softmax probability → if that probability is ≥ 0.60, show the class label and confidence; otherwise show “Uncertain”, and draw the result as a colored border and text banner on the output image (Section 5.3).

5. SYSTEM IMPLEMENTATION

5.1 Computer Vision Implementation

The demonstration pipeline is implemented as a single recognize(source, show=True) function in the notebook (Step 7). It accepts either an image file path or a BGR NumPy array (e.g. a camera frame), applies the Section 4.6 pipeline, and returns the predicted label, its confidence, and a copy of the input image annotated with the result. This keeps the training pipeline (tf.data, Section 3–4) and the single-image demo pipeline (OpenCV) using the same trained model and the same class order, so predictions from either path are directly comparable (see Section 4.2 for the one implementation difference between them).

5.2 Input and Camera Processing

Three input paths are demonstrated: (1) images already in the test split, (2) a file uploaded through the Colab file picker, and (3) a live snapshot captured from the browser's webcam via a small JavaScript helper (capture_from_webcam()) that returns a single JPEG frame. Outside Colab, the same recognize() function accepts a frame read with cv2.VideoCapture(0), so the pipeline is not tied to the Colab-only webcam helper.

5.3 Visual Output

Each recognized image is annotated with: a colored border (green if confident, orange if below the 0.60 threshold) drawn with cv2.rectangle, and a text banner in the top-left corner showing the predicted class label and confidence percentage, drawn with cv2.putText. The full per-class softmax probabilities are also printed below the image.

6. TESTING AND EVALUATION

6.1 Computer Vision Testing

The trained model is evaluated exactly once on the 464-image testing split (Section 3.5), which was never used for training or validation. model.predict() is run over the whole test set in a single call (test_ds, not shuffled, so predictions line up index-for-index with the true labels) and the argmax of each softmax output is compared against the true class.

6.2 Evaluation Metrics

Overall accuracy, per-class precision/recall/F1-score (sklearn classification_report), macro and weighted F1, and a confusion matrix (counts and row-normalized) are computed. These are the metrics the Midterm requirements call for a classification task (accuracy metrics and a confusion matrix).

6.3 Training, Validation, and Testing Results

Final test-set results for the reported run (results.json, confusion_matrix.png):

- **Test accuracy: 87.93%** (408 of 464 test images correctly classified).
- **Macro F1: 0.878**

Class	Precision	Recall	F1-score
Cardboard	0.8806	0.8551	0.8676
Glass	0.8485	0.8889	0.8682
Metal	0.9052	0.8824	0.8936
Paper	0.8353	0.9467	0.8875
Plastic	0.9000	0.8478	0.8731

Table 5. Per-class precision, recall, and F1-score on the test set (per-class F1 from results.json; precision and recall derived from confusion_matrix.png's counts, which reproduce those F1 values exactly).

This falls short of the 90% test-accuracy objective stated in Section 1.3; the gap and likely causes are discussed in Section 7.

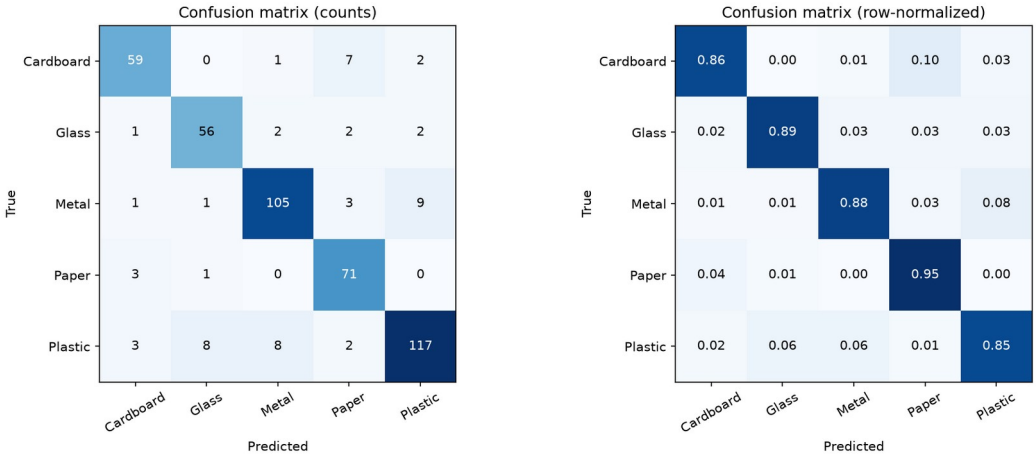


Figure 4. Confusion matrix on the 464-image test set (counts, left; row-normalized, right).

Most common mistakes (true → predicted : count): Metal → Plastic (9), Plastic → Glass (8), Plastic → Metal (8), Cardboard → Paper (7), Paper → Cardboard (3).

6.4 Sample Detection / Recognition Results

56 of the 464 test images (12.1%) were misclassified, read directly from the confusion matrix in Figure 4 (464 total minus 408 correct). Individual misclassified example images are not available for this run, since results.json and confusion_matrix.png record aggregate counts rather than per-image predictions.

6.5 Unseen Situations

The group is preparing an external cross-dataset test set, separate from RealWaste, using openly licensed public images for each of the 5 classes, organized the same way as the training data (one folder per class). This tests whether the model generalizes beyond RealWaste's own camera and lighting rather than just the held-out RealWaste test split in Section 6.3. At the time of this report the external test folders exist but are not yet populated, so no external-test accuracy is available yet; running this evaluation once the folders are filled is the clearest remaining gap before the Final stage (Section 9.2). The notebook also includes an optional cell (Step 7c) for the same kind of unseen-image test, which was not executed in the submitted notebook.

7. RESULTS AND DISCUSSION

7.1 Computer Vision Results

The MobileNetV2 transfer-learning model reaches 87.93% accuracy on the held-out RealWaste test set (5 classes, 464 images), with macro F1 0.878. All five classes score an F1 above 0.86, with no single class dragging the average down.

7.2 Analysis of Results

Paper (recall 94.7%) and Plastic (precision 90.0%) are the most reliably recognized classes. Metal is most often confused with Plastic (9 mistakes out of 464 test images), and Plastic is confused with both Glass (8) and Metal (8), consistent with those classes often containing shiny, reflective, or clear surfaces under the dataset's landfill lighting. Cardboard is most often confused with Paper (7 mistakes), which share similar color and fibrous texture, exactly the confusion the group anticipated in Section 1.4. The 87.93% result is 2.07 percentage points below the 90% objective set in Section 1.3; given that the dataset is imbalanced and visually ambiguous for several class pairs, the group considers this a reasonable result rather than a failed one, with room to close the gap through the recommendations in Section 9.2 rather than a change of approach.

7.3 Errors and Failure Cases

The dominant failure mode on the test set is confusion between visually similar material classes (Metal→Plastic, Plastic↔Glass/Metal, Cardboard→Paper; Section 6.3), not random or

unexplainable errors. Individual misclassified example images are not available for this run (Section 6.4), so this analysis is based on the aggregate confusion-matrix counts rather than example-by-example inspection.

8. PROBLEMS ENCOUNTERED AND LIMITATIONS

All training images come from a single landfill facility and camera setup, so performance on a different camera, background, or lighting is not yet measured; the external cross-dataset test described in Section 6.5 is prepared but not yet populated with images. The selected 5-class subset is imbalanced (Plastic 921 vs. Glass 420 images), addressed with class weights but not eliminated as a source of bias. The system recognizes exactly one of the five trained classes per image; an object outside these classes, or an image containing more than one item, is forced into the closest of the five rather than rejected. Individual misclassified example images for this run were not retained, which limits qualitative error analysis to the aggregate confusion matrix (Section 6.3).

9. CONCLUSION AND RECOMMENDATIONS

9.1 Conclusion

The group built and evaluated a 5-class waste-image classifier (Cardboard, Glass, Metal, Paper, Plastic) using MobileNetV2 transfer learning on the RealWaste dataset, satisfying the Midterm requirements for Specialization 8 (Object Sorting): at least 4 classes with at least 100 images each, a prepared train/validation/test split, image preprocessing and augmentation, a working CV model, and evaluation with accuracy metrics and a confusion matrix. The final test accuracy is 87.93% (macro F1 0.878), short of the group's internal 90% target but demonstrating a working SEE → RECOGNIZE pipeline with a visible label and confidence output.

9.2 Recommendations

- **Populate and run the external cross-dataset test.** Add openly licensed public images to the prepared external_test/ class folders and evaluate the model on them to get a measured generalization accuracy before the Final stage, rather than relying on the RealWaste test split alone (Section 6.5).

- **Target the Metal/Plastic and Cardboard/Paper confusions.** Consider additional training images or augmentation that emphasizes the cues that distinguish these pairs (e.g., surface reflectance, texture) to close the gap to the 90% objective.

- **Planned for the Final project: robot integration.** The recognized class will feed a decision step that chooses a sorting destination, driving a 4WD chassis's motors through a motor driver to physically move the item (the SEE → DECIDE → ACT loop), to be

demonstrated in the Test 1 (Figure-8 line tracking) and Test 2 (Object Sorting) evaluations. None of this is in scope for this Midterm submission.

REFERENCES

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems* [Computer software]. https://www.tensorflow.org/

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Chollet, F. (2015). *Keras* [Computer software]. https://keras.io

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248–255). IEEE. https://doi.org/10.1109/CVPR.2009.5206848

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Python Software Foundation. (n.d.). *Python* [Computer software]. https://www.python.org/

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). IEEE. https://doi.org/10.1109/CVPR.2018.00474

Single, S., Iranmanesh, S., & Raad, R. (2023). *RealWaste* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5SS4G

Single, S., Iranmanesh, S., & Raad, R. (2023). RealWaste: A novel real-life data set for landfill waste classification using deep learning. *Information, 14*(12), Article 633. https://doi.org/10.3390/info14120633